

CPP MODÜLÜ 04 (/PROJECTS/CPP-MOD PROJESİ İÇİN ÖLÇEK (/PROJECTS/CPP-MC

Bu takımdaki 1 öğrenciyi değerlendirmelisiniz.



Git deposu

git@vogosphere.42istanbul.com.tr:vogosphere/intra-uuid-6fef65

giriş

Lütfen aşağıdaki kurallara uyun:

- Değerlendirme süreci boyunca kibar, saygılı ve yapıcı olun . Topluluğun iyiliği buna bağlıdır.
- Çalışması değerlendirilen öğrenci veya grupla projelerindeki olası aksaklıkları belirleyin. Belirlenen sorunları tartışmak ve değerlendirmek için zaman ayırın. - Akranlarınızın projenin işlevselliğinin kapsamını nasıl anladıkları konusunda bazı farklılıklar olabileceğini göz önünde bulundurmalısınız . Her z ve onları olabildiğince dürüst bir şekilde değerlendirin . Pedagoji, ancak ve ancak akran değer yapıldığında faydalıdır.

Yönergeler

- Yalnızca değerlendirilen öğrenci veya grubun Git deposunda teslim edilen çalışmayı notland
- Git deposunun öğrenci(ler)e ait olduğundan emin olun. Projenin beklenen proje olduğundan emin olun. Ayrıca, 'git clone' komutunun boş bir klasörde kullanıldığından emin olun. - Sizi kandırmak ve resmi deponun içeriği olmaya değerlendirmenizi sağlamak için kötü amaçlı takma adların kullanılmadığından emin olun . - Herhangi bir sürpriz mümkünse, notlandırmayı kolaylaştırmak için kullanılan tüm komut dosyalarını (test veya otom dosyaları) birlikte gözden geçirin. - Değerlendireceğiniz ödevi tamamlamadıysanız, değerlend başlamadan önce konunun tamamını okumalısınız. - Boş bir depo, çalışmayan bir program, N durumları bildirmek için mevcut bayrakları kullanın . Bu durumlarda, değerlendirme işlemi son veya hile durumunda -42 olur. Ancak, kopya çekme durumu hariç, öğrencilerin gelecekte tekr hataları belirlemek için teslim edilen çalışmaları birlikte gözden geçirmeleri şiddetle tavsiye ed dosyası (varsa) dışında hiçbir dosyayı düzenlememelisiniz . Bir dosyayı düzenlemek istiyorsanız öğrenciyle nedenlerini açıkça belirtin ve ikinizin de bu konuda hemfikir olduğundan emin olun. sızıntılarının olmadığını da doğrulamalısınız. Yığın üzerinde ayrılan tüm bellek, yürütmenin so bir şekilde serbest bırakılmalıdır. Bilgisayarda bulunan farklı araçlardan herhangi birini (örneğin e_fence) kullanabilirsiniz . Bellek sızıntısı durumunda, ilgili bayrağı işaretleyin.

 konu.pdf (<https://cdn.intra.42.fr/pdf/pdf/197137/en.subject.pdf>)

Ön testler

Hile yapıldığından şüpheleniliyorsa, değerlendirme burada durdurulur. Hile bildirimini için "Hile" kararı sakın ve akıllıca verin ve lütfen bu düğmeyi dikkatli kullanın.

Önkoşullar

Kodun C++ ile ve -Wall, -Wextra, -Werror bayraklarıyla derlenmesi gerekiyor. Bu projenin C++98 standardına uyması gerektiğini unutmayın. Bu nedenle, C++11 (ve sonrası) fonksiyonları veya konteynerleri BEKLENMEMEKTEDİR.

Bunlardan herhangi biri, söz konusu alıştırmayı değerlendirmeniz gerektiği anlamına gelir:

- Bir fonksiyon, (şablon fonksiyonları hariç) bir başlık dosyasında uygulanır.
- Bir Makefile, gerekli bayraklar olmadan ve/veya C++'tan farklı bir derleyiciyle derleme y

Bunlardan herhangi biri, projeyi "Yasaklanmış

Fonksiyon" olarak işaretlemeniz gerektiği anlamına gelir:

- "C" fonksiyonlarının kullanımı (*alloc, *printf, free).
- Egzersiz yönergelerinde izin verilmeyen bir işlevin kullanılması.
- "using namespace <ns_name>" veya "friend" anahtar kelimesinin kullanımı.
- Use of an external library, or features from versions other than C++98.

 Yes

 No

Exercise 00: Polymorphism

As usual, there has to be enough tests to prove the program works as expected. If there isn't, exercise. If any non-interface class is not in orthodox canonical class form, do not grade this e

Makefile

There is a Makefile that compiles using the appropriate flags.

If this is not the case, the evaluation of this exercise stops here. you can move on to the next e

 Yes

 No

First check

There is an Animal class that has one attribute:

One string called type.

You must be able to instantiate and use this class.

 Yes

 No

Inheritance

They are at least two classes that inherit from Animal: Cat and Dog.

The constructor and destructor outputs must be clear.

Ask the student about constructor and destructor orders.

 Yes

 No

Easy derived class

The attribute type is set to the appropriate value at creation for every animal. Cat must have "Cat" and Dog must have "Dog".

 Yes

 No

Animal

Using makeSound() function always called the appropriate makeSound() function. makeSound() should be virtual! Verify it in the code.
virtual void makeSound() const;
The return value is not important but virtual keyword is mandatory.

There should be an example with a WrongAnimal and WrongCat that don't use the virtual keyword (see subject).
The WrongCat must output the WrongCat makeSound() only when used as a wrongCat.

☒ Yes☐ No

Exercise 01: I do not want to set the world on fire

As usual, there has to be enough tests to prove the program works as expected. If there isn't, exercise. If any non-interface class is not in orthodox canonical class form, do not grade this exercise.

Makefile

There is a Makefile that compiles using the appropriate flags.
If this is not the case, the evaluation of this exercise stops here. you can move on to the next exercise.

☒ Yes☐ No

Concrete Animal

There is a new class called Brain.
Cat and Dog have the required private Brain attribute.
The Brain attribute should not be inside the Animal class.
The Brain class has specific outputs upon creation and deletion.

☒ Yes☐ No

Concrete Brain

The copy a Cat or a Dog should be a deep copy.
Test something like:
Dog basic;
{
Dog tmp = basic;
}
If the copy is shallow, tmp and basic will use the same Brain and the Brain will get deleted with tmp at the end of the scope.
The copy constructor should do a deep copy too.
That's why a clean implementation in orthodox canonical form will save you from hours of pain.

☒ Yes☐ No

Destruction chaining

The destructors in Animal must be virtual.
Ask an explanation of what will happen without the virtual keyword over Animal destructor.
Test it.

☒ Yes☐ No

Assignment and copy

The copy and assignment behaviors of the Cat and Dog are like the subject requires.
Deep copy means you need to create a new Brain for the Cat or Dog.
Check that the canonical form is really implemented (i.e. no empty copy assignment operators and so forth). Nothing should be public for no reason. Moreover, this code is very simple so it needs to be clean!

☒ Yes☐ No

Main function tests

The main function must implement the tests explicitly required in the subject:

- Create and fill an array of Animal objects.
- Half of them must be Dogs, the other half Cats.
- At the end of the program, loop over this array and delete every Animal directly as an Animal.

- The appropriate destructors must be called in the expected order.
- Check that there are no memory leaks.

✓ Yes

✗ No

Exercise 02: Abstract class

As usual, there has to be enough tests to prove the program works as expected. If there isn't, exercise. If any non-interface class is not in orthodox canonical class form, do not grade this exercise.

Makefile

There is a Makefile that compiles using the appropriate flags.

If this is not the case, the evaluation of this exercise stops here. you can move on to the next exercise.

✓ Yes

✗ No

Abstract class

There is an Animal class exactly like the one in the subject.

The Animal::makeSound is a pure virtual function.

It should look like : virtual void makeSound() const = 0;

The "= 0" part is mandatory.

You should not be able to instantiate an Animal.

Animal test; //should give you a compile error about the class being abstract

✓ Yes

✗ No

Concrete Animal

Class Cat and Dog are still present and work exactly like in ex01.

✓ Yes

✗ No

Exercise 03: Interface and recap

As usual, there has to be enough tests to prove the program works as expected. If there isn't, exercise. If any non-interface class is not in orthodox canonical class form, do not grade this exercise.

Makefile

There is a Makefile that compiles using the appropriate flags.

If this is not the case, the evaluation of this exercise stops here. you can move on to the next exercise.

✓ Yes

✗ No

Interfaces

There are ICharacter and IMateriaSource interfaces that are exactly like required in the subject.

✓ Yes

✗ No

MateriaSource

The MateriaSource class is present and implements IMateriaSource. The member functions work as intended.

✓ Yes

✗ No

Concrete Materia

There are concrete Ice and Cure classes that inherit from AMateria. Their clone() method is correctly implemented. Their outputs are correct.

The AMateria class is still abstract (clone() is a pure function).

virtual ~AMateria() is present.

AMateria contains a protected string attribute to store the type.

✓ Yes

✗ No

Character

The Character class is present and implements ICharacter. It has an inventory of 4 Materias maximum.
The member functions are implemented as the subject requires.
The copy and assignment of a Character are implemented as required (deep copy).

✔ Yes ✖ No

Ratings

Don't forget to check the flag corresponding to the defense

✔ Ok ★ Outstanding project

- Empty work
- 📄 Incomplete work
- ⚙ Invalid compilation
- 📄 Cheat
- ⚠ Concerning situation
- 💧 Leaks
- 🚫 Forbidden function
- 💬 Can't sup

Conclusion

Bu değerlendirmeye yorum bırakın (en fazla 2048 karakter)

Değerlendirmeyi bitir